LEARN BY BUILDING

Docker, CI/CD & DevOps

A practical 12-week roadmap from containers to reliable production delivery.

Use the OptiMind project as your running laboratory: containerize it, test it, deploy it, monitor it, and improve it.

October 2026 • Practical learning plan

How to use this roadmap

Study the concept, complete the lab, and write down what you learned. Each week should end with something that runs and can be demonstrated.

- Spend 60-90 minutes learning and 60-90 minutes building on most days.
- Keep every lab in Git with a README, a diagram, and a rollback note.
- Prefer small, repeatable automation over manual server work.
- Do not move to the next stage until you can explain the current stage to someone else.

Target outcome: by the end, one commit can run tests, build an image, publish an artifact, deploy safely, and provide enough logs and health checks to diagnose failures.

The learning path

Weeks	Focus	Deliverable
1-2	Linux, Git, networking	A documented Ubuntu lab server
3-4	Docker fundamentals	OptiMind running with Compose
5-6	Images, registries, security	Small, reproducible production images
7-8	CI/CD pipelines	Automated test, build, and deploy pipeline
9-10	Infrastructure and observability	Health checks, logs, metrics, alerts
11-12	Reliability and capstone	Documented zero-downtime release

Phase 1 - Foundations (Weeks 1-2)

Learn enough Linux and networking to understand where an application is running and why a connection succeeds or fails.

Study

- Processes, services, systemd, users, groups, permissions, SSH, files, and logs.
- TCP/IP basics, ports, DNS, HTTP, localhost versus a network IP, and firewalls.
- Git branches, commits, pull requests, environment variables, and secrets.

Lab

```
sudo systemctl status docker
ip addr
ss -ltnp
curl http://localhost:5173
git log --oneline -5
```

Definition of done

- You can explain the path from a browser request to a container and identify the port at each step.

Phase 2 - Docker fundamentals (Weeks 3-4)

Understand images, containers, networks, volumes, Dockerfiles, and Compose.

Study

- Dockerfile instructions: FROM, WORKDIR, COPY, RUN, ENV, EXPOSE, and CMD.
- The difference between an image and a container; container lifecycle and logs.
- Compose services, depends_on, health checks, environment variables, and volumes.

Lab

```
docker build -t optimind-backend -f backend/Dockerfile .
docker compose up -d --build
docker compose ps
docker compose logs -f backend
docker compose down
```

Definition of done

- You can rebuild OptiMind from a clean checkout and explain what data survives docker compose down.

Phase 3 - Images, registries, and security (Weeks 5-6)

Make images smaller, safer, and repeatable.

- Use .dockerignore, multi-stage builds, pinned versions, non-root users, and minimal base images.
- Understand image tags, digests, registries, vulnerability scanning, and secret handling.
- Separate build-time configuration from runtime configuration; never bake passwords into images.

Lab

```
docker image ls
docker history optimind-backend
docker inspect optimind-backend
docker compose config --quiet
```

Security rule: environment files and secrets stay outside Git. A Docker image should be safe to share without exposing credentials.

Phase 4 - CI/CD (Weeks 7-8)

CI/CD turns a code change into a tested, reviewable, repeatable release.

Continuous Integration

- Run linting, unit tests, type checks, and a frontend build on every pull request.
- Fail fast, keep logs readable, cache dependencies, and publish test results.

Continuous Delivery

- Build an immutable image after tests pass, tag it with the commit SHA, and publish it to a registry.
- Deploy to staging automatically; require an approval for production if the risk warrants it.
- Use health checks, a migration step, and a rollback procedure.

Example pipeline stages

```
checkout -> install -> lint/test -> build -> scan -> publish -> deploy -> smoke test
```

Lab

- Create a CI workflow for OptiMind that runs the existing npm build and backend pytest suite, then builds both Docker images.

Starter GitHub Actions pipeline

```
name: CI
on:
  pull_request:
  push:
    branches: [main]
jobs:
  test-build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: {node-version: 20, cache: npm}
      - run: npm ci --prefix frontend
      - run: npm run build --prefix frontend
      - uses: actions/setup-python@v5
        with: {python-version: '3.12'}
      - run: pip install -r backend/requirements.txt
      - run: pytest -q backend/tests
```

Free places to run it

Option	Best for	Notes
GitHub Actions	Public repos and learning	Hosted runners; free usage is limited for private repositories
GitLab CI/CD	A complete GitLab workflow	Free shared runner minutes; bring your own runner for more control
Jenkins	A fully self-hosted lab	Free software; run it on your Ubuntu Docker server
Gitea or Woodpecker	Private, self-hosted Git	Free software; you provide the server and maintenance

Recommended starting setup: GitHub Actions for CI plus your Ubuntu Docker machine as the deployment target. It costs nothing for a small public project; private repositories have usage limits and the server still consumes electricity.

Phase 5 - Infrastructure and observability (Weeks 9-10)

Learn how to operate the system after deployment.

- Infrastructure as code concepts: declared state, reviewable changes, drift, and repeatability.
- Logs: structured messages, correlation IDs, retention, and useful error context.
- Metrics: request rate, latency, error rate, container restarts, disk, memory, and model latency.
- Health checks, readiness versus liveness, alert thresholds, and incident notes.

Lab

```
docker stats
docker compose ps
curl http://localhost:8000/api/v1/status
docker compose logs --since=10m
```

Phase 6 - Reliability and capstone (Weeks 11-12)

Bring the skills together in one production-style release process.

Capstone project

Deploy OptiMind on an Ubuntu office server with a static IP or internal hostname. The finished system should have:

- Docker Compose with persistent PostgreSQL and Ollama volumes.
- Automatic startup after reboot and a documented backup/restore process.
- A CI pipeline that tests and builds every change.
- A staging or test environment, a smoke test, and a rollback command.
- A dashboard or at least a repeatable status and log-check procedure.
- An internal hostname with firewall rules that expose only the frontend.

Release checklist

Check	Evidence
Build	Images build from a clean checkout
Test	CI passes lint, tests, and frontend build
Deploy	Compose starts and migrations complete
Health	Status endpoint and smoke test pass
Rollback	Previous image/tag can be restored
Security	Secrets are not in Git or images

Your goal is not to memorize Docker commands. Your goal is to make delivery predictable: build, test, deploy, observe, recover.

Recommended daily practice

- Read one short concept and immediately run one command.
- Break something safely, then recover it using logs and documentation.
- Write a one-paragraph incident note for every failure: symptom, cause, fix, prevention.
- Keep a personal command reference, but always understand what each command changes.

Once this roadmap is complete, the next specialization can be Kubernetes, Terraform, cloud networking, or platform engineering - choose based on the kind of systems you want to operate.